Tagless Final

User registration against UserRepo/Email/Logger algebras; IO + pure t...

Season 7 · Functional · Advanced

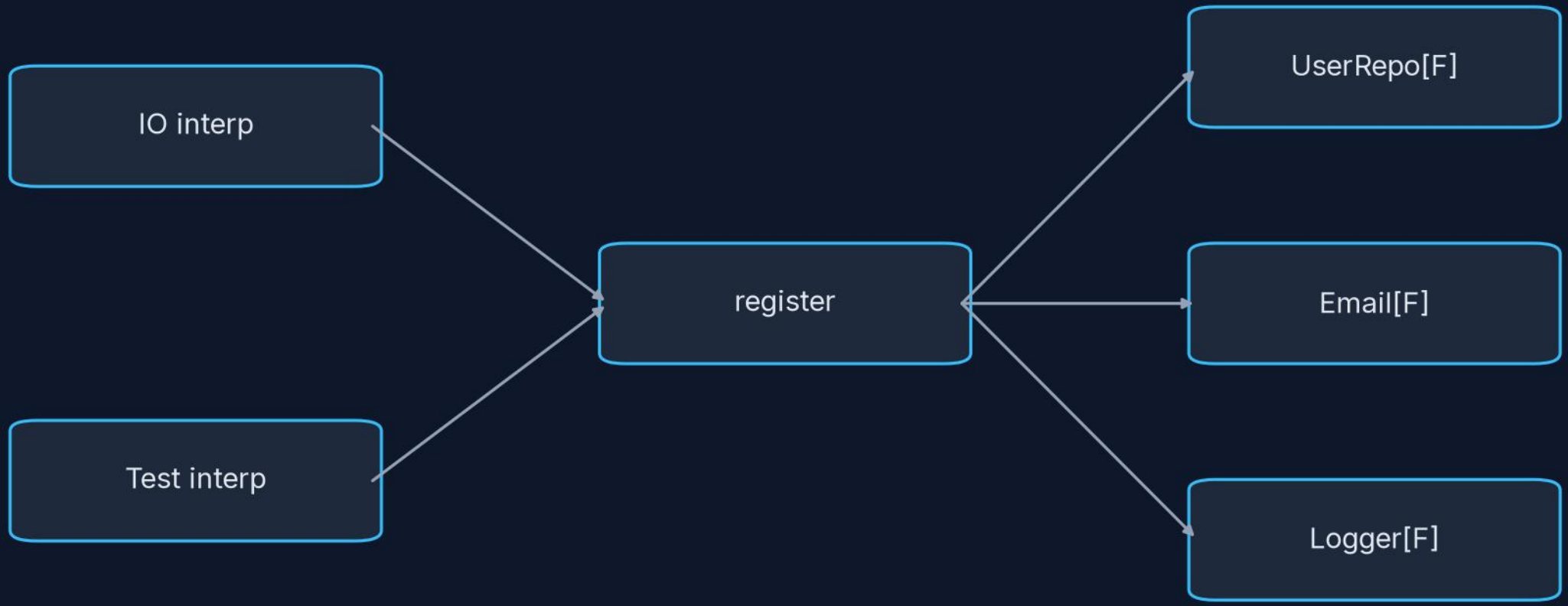
Today's Hook

- Problem: User registration against UserRepo/Email/Logger algebras; IO + pure t...
- Naive pain cue: Failure path is asserted in ProblemTest (injected fault / invalid input)
- We will refactor to Tagless Final with a runnable Demo.
- Failure path is locked by ProblemTest.

The Problem

- Register a user: persist User(id,email), send welcome email, log steps.
- Program is written once against UserRepo[F], EmailSender[F], Logger[F].
- Naive hard-wires Map + System.out — cannot assert emails without capturing stdout.
- Success: same register() under IoUserRepo and TestState; duplicate email skips second welcome.

Structure



Tagless Final: User registration against UserRepo/Email/Logger algebr...

Real-World Scenario

- Inputs: id + email address; Env is the effect dictionary F.
- Happy path: save user, send Welcome, append log lines.
- Failure/edge: duplicate email → log 'duplicate', no second send.
- Test cake: TestState.emails / .logs / .users inspected after runWith(state).

Naive Code

```java
package com.buildwithwaqar.patterns.functional.tagless.problem;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

/** Naive: registration hard-wires Map DB + System.out email — untestable without mocks frameworks. */
public final class NaiveRegistration {
    private final Map<String, Algebras.User> db = new LinkedHashMap<>();
    private final List<String> outbox = new ArrayList<>();

    public Algebras.User register(String id, String email) {
        if (db.containsKey(email)) {
            System.out.println("dup " + email); // side effect baked in
            return db.get(email);
        }
        var u = new Algebras.User(id, email);
        db.put(email, u);
        outbox.add("welcome->" + email); // cannot swap for test assertion easily in unit style
        System.out.println("sent welcome to " + email);
        return u;
    }
```

Naive Code (2)

```
    public List<String> outbox() { return List.copyOf(outbox); }
}
```

Why Naive Hurts

- NaiveRegistration news a Map and prints email — untestable without a mock framework.
- Duplicate email path still couples to System.out.
- Swapping 'prod' for 'test' means rewriting the service.
- Effect choice (IO vs pure state) is baked into call sites.

Solution

```java
package com.buildwithwaqar.patterns.functional.tagless.problem;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.function.Function;

/**
 * Tagless Final algebras for user registration.
 * Programs are written once against UserRepo/EmailSender/Logger; interpreters choose the effect.
 */
public final class Algebras {
    private Algebras() {}

    public record User(String id, String email) {}

    /** Minimal "effect" wrapper — either Identity (prod sync) or accumulating TestState. */
    public interface Kind<F, A> {
        A unsafeRun();
    }
```

Solution (2)

```
public interface Monad<F> {
    <A> Kind<F, A> pure(A a);
    <A, B> Kind<F, B> flatMap(Kind<F, A> fa, Function<A, Kind<F, B>> f);
    default <A, B> Kind<F, B> map(Kind<F, A> fa, Function<A, B> f) {
        return flatMap(fa, a -> pure(f.apply(a)));
    }
}

public interface UserRepo<F> {
    Kind<F, Optional<User>> findByEmail(String email);
    Kind<F, User> save(User user);
}

public interface EmailSender<F> {
    Kind<F, Void> send(String to, String subject, String body);
}

public interface Logger<F> {
    Kind<F, Void> info(String msg);
}
```

Solution (3)

```
/** Shared registration program — no concrete effect type. */
public static <F> Kind<F, User> register(
        Monad<F> M, UserRepo<F> users, EmailSender<F> email, Logger<F> log,
        String id, String address) {
    return M.flatMap(log.info("register " + address), ignored ->
            M.flatMap(users.findByEmail(address), existing -> {
                if (existing.isPresent()) {
                    return M.flatMap(log.info("duplicate " + address), x ->
                            M.pure(existing.get())); // return existing — caller may treat as failure
                }
                var user = new User(id, address);
                return M.flatMap(users.save(user), saved ->
                        M.flatMap(email.send(address, "Welcome", "Hi " + id), sent ->
                                M.flatMap(log.info("registered " + id), y -> M.pure(saved))));
            }));
}

// ---- Identity / "IO-style" interpreter (eager, side-effecting) ----
public enum IoWitness { W }
public record Io<A>(A value) implements Kind<IoWitness, A> {
    public A unsafeRun() { return value; }
}
```

Solution (4)

```
public static final Monad<IoWitnes
```

Runnable main

```java
package com.buildwithwaqar.patterns.functional.tagless.problem;

/** Problem: same registration program under IO-style and pure test interpreters. */
public final class ProblemDemo {
    public static void main(String[] args) {
        System.out.println("=== Tagless Final: user registration algebras ===");
        System.out.println("--- naive (hard-wired Map + System.out) ---");
        var naive = new NaiveRegistration();
        System.out.println(naive.register("u1", "a@b.c"));
        System.out.println("outbox=" + naive.outbox());

        System.out.println("--- IO-style interpreter ---");
        var repo = new Algebras.IoUserRepo();
        var mail = new Algebras.IoEmail();
        var log = new Algebras.IoLogger();
        var user = Algebras.register(Algebras.IO, repo, mail, log, "u2", "x@y.z").unsafeRun();
        System.out.println("user=" + user);
        System.out.println("emails=" + mail.sent());
        System.out.println("logs=" + log.lines());

        System.out.println("--- pure test interpreter (shared state) ---");
        var state = Algebras.TestState.empty();
        var prog = Algebras.register(
                Algebras.TEST, new Algebras.TestUserRepo(), new Algebras.TestEmail(), new Algebras.TestLogger(),
                "u3", "t@est.io");
        var saved = ((Algebras.Test<Algebras.User>) prog).runWith(state);
        System.out.println("user=" + saved);
        System.out.println("emails=" + state.emails());
        System.out.println("logs=" + state.logs());
        System.out.println("db=" + state.users().keySet());
    }
```

Runnable main (2)

```
}
```

Walkthrough Output

```
=== Tagless Final: user registration algebras ===
--- naive (hard-wired Map + System.out) ---
sent welcome to a@b.c
User[id=u1, email=a@b.c]
outbox=[welcome->a@b.c]
--- IO-style interpreter ---
user=User[id=u2, email=x@y.z]
emails=[x@y.z|Welcome|Hi u2]
logs=[register x@y.z, registered u2]
--- pure test interpreter (shared state) ---
user=User[id=u3, email=t@est.io]
emails=[t@est.io|Welcome]
logs=[register t@est.io, registered u3]
db=[t@est.io]
```

Step-by-Step Trace

- Match stdout to stages — look for: sent welcome to a@b.c
- Then the injected failure: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Re-run: java -cp … 59-tagless-final ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```java
package com.buildwithwaqar.patterns.functional.tagless.problem;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.function.Function;

/**
 * Tagless Final algebras for user registration.
 * Programs are written once against UserRepo/EmailSender/Logger; interpreters choose the effect.
 */
public final class Algebras {
    private Algebras() {}

    public record User(String id, String email) {}

    /** Minimal "effect" wrapper — either Identity (prod sync) or accumulating TestState. */
    public interface Kind<F, A> {
        A unsafeRun();
    }
```

Modern Java 25 / Zazr (2)

```
public interface Monad<F> {
    <A> Kind<F, A> pure(A a);
    <A, B> Kind<F, B> flatMap(Kind<F, A> fa, Function<A, Kind<F, B>> f);
    default <A, B> Kind<F, B> map(Kind<F, A> fa, Function<A, B> f) {
        return flatMap(fa, a -> pure(f.apply(a)));
    }
}

public interface UserRepo<F> {
    Kind<F, Optional<User>> findByEmail(String email);
    Kind<F, User> save(User user);
}

public interface EmailSender<F> {
    Kind<F, Void> send(String to, String subject, String body);
}
```

Trade-offs

- Use Tagless Final when «User registration against UserRepo/Email/Logger algebras; IO + pure test interpreters» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Tagless Final in product language.

Related Patterns

- Name the pattern usually confused with Tagless Final and separate the forces.
- Sketch the roles for «User registration against UserRepo/Email/Logger algebras; IO + pure test interpreters» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.